

المدرسة الحسنية للأشغال العمومية
ⵜⵉⵎⵔⴰⵣⵉⵏ ⵜⵓⵙⵙⴰⵏⵉⵜ ⵏ ⵜⵎⵓⵔⵜ ⵜⴰⵎⴰⵡⴰⵙⵜ
ECOLE HASSANIA DES TRAVAUX PUBLICS

Course

CRAG from first principles

Everything used in the project, explained from zero: computers, models, retrieval, graphs, engineering

Presented by	Daoudi Ahmed Khalil
Supervised by	Dr. Abdelali El Bouchti
Degree	Master in Data Science and Engineering (MDSE)
Institution	Ecole Hassania des Travaux Publics
Academic year	2025/2026

How to read this course

This course assumes you know nothing. By the end you should be able to explain every part of CRAG to a jury, rebuild it, and change it with confidence. Each chapter builds on the previous one, so read them in order. Each ends with a few self-check questions; the answers are at the end of the course.

Three conventions:

- Words in **bold** are defined where they first appear and collected in the glossary at the end.
- `Monospace` is code, a file or a command.
- Boxes marked **In CRAG** point to the exact place in the code where the idea is used.

The course follows the path of the project: first the foundations (computers, Python, the web), then machine learning and language models, then retrieval, graphs, and how CRAG combines them, then the engineering that makes it a product: API, interface, tests, evaluation, deployment.

Table of contents

How to read this course 2

Computers, data and text 6
- Everything is numbers 6
- Files and formats 6
- Self-check 6

Python and the tools around it 7
- Programs, functions and modules 7
- Libraries, pip and virtual environments 7
- Environment variables and secrets 7
- Self-check 7

The web: HTTP and APIs 8
- Clients, servers and requests 8
- APIs 8
- Rate limits and retries 8
- Self-check 8

Machine learning in one chapter 9
- Learning a function from examples 9
- Loss and gradient descent 9
- Neural networks 9
- Self-check 9

Large language models 10
- Tokens 10
- Predicting the next token 10
- The Transformer and attention 10
- Temperature 10
- Mixture of experts and reasoning models 10
- What LLMs cannot do alone 11
- Self-check 11

Prompting 12
- System and user messages 12
- Asking for structured output 12
- Grounding 12
- Self-check 12

Embeddings and vector search 13
- Meaning as a direction 13
- Measuring closeness: the cosine 13
- Queries and passages 13
- Finding nearest vectors fast: HNSW 13

Self-check 14

Retrieval-Augmented Generation 15
The idea 15
Chunking 15
The limits of classic RAG 15
Self-check 15

Knowledge graphs 16
Nodes, edges, triples 16
Hops and breadth-first search 16
Property graphs and Neo4j 16
Self-check 17

Building the graph with an LLM 18
Extraction 18
Never trust, always validate 18
Entity resolution 18
Storing 18
Self-check 18

Optical character recognition 19
Self-check 19

GraphRAG: how CRAG retrieves 20
1. Vector ranking 20
2. Entity linking 20
3. Graph expansion 20
4. Graph ranking 20
5. Fusion with Reciprocal Rank Fusion 20
The corrective loop 21
Self-check 21

LangGraph: the workflow as a state machine 22
State machines 22
LangGraph 22
CRAG's workflow 22
Self-check 22

Experts, recommendations and explanations 23
Ranking experts 23
Recommending documents 23
Explaining answers 23
Self-check 24

The API and the interface 25
FastAPI 25
Security basics 25
The web page 25

Self-check 25

Testing 26

Self-check 26

Evaluation 27

A fair comparison 27

Metrics 27

The three benchmarks 27

Self-check 27

Git, Docker and deployment 28

Git and GitHub 28

Docker 28

Serverless deployment 28

Self-check 28

Putting it all together 29

Glossary 31

Answers to the self-checks 33

Chapter 1

Computers, data and text

Everything is numbers

A computer stores and manipulates only numbers, in binary: sequences of 0 and 1 called **bits**. Eight bits make a **byte**, which can represent 256 different values (2^8). Everything else (text, images, programs) is an agreed way of interpreting numbers.

Text is stored with an **encoding** that maps each character to a number. The modern standard, **UTF-8**, gives "A" the number 65 and uses several bytes for characters such as "é" or Arabic letters. When you open a file and see strange symbols, the program used the wrong encoding.

An **image** is a grid of **pixels**; each pixel is three numbers (red, green, blue) from 0 to 255. A scanned page is just such a grid: it contains no letters a computer can read until **OCR** (Chapter 13) finds them.

Files and formats

A **PDF** file is a description of pages: where to draw each character, line and image. A "digital" PDF (exported from Word) contains a **text layer**, so the characters can be extracted. A "scanned" PDF contains only images of pages. This difference decides whether CRAG needs OCR.

JSON (JavaScript Object Notation) is a text format for structured data, readable by humans and programs:

```
{"name": "Alice", "skills": ["Python", "SQL"], "years": 5}
```

Curly braces hold **objects** (key–value pairs), square brackets hold **lists**. CRAG asks the language model to answer in JSON so that the code can read the answer reliably.

Self-check

1. Why can a computer not read the words of a scanned page directly?
2. What is the difference between `{...}` and `[...]` in JSON?

Chapter 2

Python and the tools around it

Programs, functions and modules

A **program** is a list of instructions. **Python** is the language CRAG is written in, chosen because the AI ecosystem is built around it. The basic unit is the **function**: a named piece of code that takes inputs and returns an output.

```
def cosine(a, b):
    return sum(x * y for x, y in zip(a, b))
```

Functions are grouped in files called **modules** (`crag/llm.py` is a module), and modules in a **package** (the `crag/` folder). A **class** groups data and the functions that act on it; `MemoryStore` and `Neo4jStore` in `crag/store.py` are classes.

Libraries, pip and virtual environments

A **library** is code written by others that you reuse: NumPy for numbers, FastAPI for web servers, LangGraph for workflows. **pip** installs libraries. A **virtual environment** (`.venv`) is a private folder of libraries for one project, so two projects can use different versions without conflict. `requirements.txt` lists the exact versions CRAG needs; pinning versions makes the installation reproducible.

Environment variables and secrets

Some settings must not be written in the code: passwords and **API keys** (secret strings that identify you to a paid service). They are stored in **environment variables**, values the operating system gives to a program when it starts. Locally, CRAG reads them from a file named `.env`, which is excluded from Git so it is never published.

> **In CRAG.** `crag/config.py` reads every setting (model names, chunk size, Neo4j password, API key) from environment variables with safe defaults. The same code therefore runs on your laptop, in Docker and on Vercel.

Self-check

1. Why is the API key in `.env` and not in `config.py`?
2. What problem does a virtual environment solve?

Chapter 3

The web: HTTP and APIs

Clients, servers and requests

When your browser opens a page, it acts as a **client** and sends a **request** to a **server**, which returns a **response**. The rules of that conversation are **HTTP**. A request has a **method** (GET to read, POST to send data), a **URL** (the address, such as `/api/ask`), **headers** (metadata, such as an authorisation key) and sometimes a **body** (the data, often JSON). A response has a **status code**: 200 means success, 4xx means the client made a mistake (401 not authorised, 404 not found, 422 invalid input, 429 too many requests), 5xx means the server failed.

APIs

An **API** (Application Programming Interface) is a set of URLs a program can call to use a service. CRAG both *uses* an API (NVIDIA's, to reach the models) and *offers* one (its own, used by the web interface). A **REST** API organises these URLs around resources and HTTP methods.

Calling the NVIDIA chat API looks like this:

```
POST https://integrate.api.nvidia.com/v1/chat/completions
Authorization: Bearer <your key>
{"model": "nvidia/nemotron-3.5-lightning-30b-a3b",
 "messages": [{"role": "user", "content": "Say OK"}]}
```

The response contains the model's reply in `choices[0].message.content`.

Rate limits and retries

Services limit how many requests you may send per minute; beyond that they answer **429**. A robust client waits and retries, doubling the wait each time (1, 2, 4, 8 seconds): this is **exponential back-off**.

> **In CRAG.** `crag/llm.py` sends every model request through one function, `_post`, that retries on 429 and 5xx errors with exponential back-off.

Self-check

1. What does status 422 mean, and when does CRAG return it?
2. Why double the waiting time between retries instead of retrying immediately?

Chapter 4

Machine learning in one chapter

Learning a function from examples

Classic programming writes rules by hand. **Machine learning** instead chooses a function from a family of functions by looking at examples. Take predicting a house price y from its surface x with a line $y = w \cdot x + b$. The numbers w and b are **parameters**. Learning means finding the parameters that make predictions close to the true prices of known examples.

Loss and gradient descent

We measure how wrong the model is with a **loss**, for example the average squared error between predictions and true values. To reduce it, we compute the **gradient**: the direction in which each parameter should move to increase the loss, and we move the opposite way by a small step. Repeating this millions of times is **gradient descent**, and this repetition is **training**. Using the trained model on new inputs is **inference**. CRAG never trains a model; it only performs inference through an API.

Neural networks

A **neuron** computes a weighted sum of its inputs and passes it through a simple non-linear function. A **layer** is many neurons side by side; a **neural network** stacks layers. With enough layers and parameters, networks can approximate very complex functions, such as "the probability of each next word given the previous ones". Training them is still gradient descent, on billions of parameters.

Self-check

1. What is the difference between training and inference? Which one does CRAG do?
2. What does the loss measure?

Chapter 5

Large language models

Tokens

A language model does not see letters or words but **tokens**: frequent pieces of words, each with an id number. "knowledge" may be one token, "Hassania" several. A model's limits (input size, price) are counted in tokens; in English, one token is about four characters.

Predicting the next token

A **large language model (LLM)** is a neural network with billions of parameters trained on huge amounts of text to predict the next token. To generate an answer, it predicts a probability for every possible next token, picks one, appends it, and repeats. Predicting text well at that scale forces the model to learn grammar, facts and patterns of reasoning. After this pre-training, models are further trained to follow instructions and be helpful.

The Transformer and attention

Modern LLMs use the **Transformer** architecture (Vaswani et al., 2017). Its core is **attention**: for each token, the model computes how much every other token in the input matters to it, and mixes their information accordingly. In "The engineer who designed the bridge said *it* was safe", attention helps link "it" to "bridge".

Temperature

The model outputs probabilities; **temperature** controls how they are used. At 0, it always picks the most likely token: the output is (nearly) deterministic. Higher values flatten the probabilities and make output more varied. CRAG uses 0 for extraction and grading (we want repeatable JSON) and 0.2 for answers.

Mixture of experts and reasoning models

A **Mixture-of-Experts (MoE)** model has, in each layer, several sub-networks called experts and a small router that sends each token to only a few of them. CRAG's chat model, Nemotron 3.5 Lightning 30B-A3B, has 30 billion parameters but uses only about 3 billion per token, which makes it fast.

A **reasoning model** can write hidden "thinking" before answering, which helps hard problems but costs time. Our tasks are short and structured, so CRAG turns thinking off with the option `enable_thinking: false`.

What LLMs cannot do alone

1. They know nothing after their training date and nothing about your private documents.
2. They **hallucinate**: they can produce fluent statements that are false, because they generate plausible text, not verified text.
3. Their input (the **context window**) is limited, and they use the middle of long inputs poorly ("lost in the middle").

The rest of this course is about solving these three problems.

Self-check

1. Why does CRAG set temperature to 0 for extraction?
2. What does "30B-A3B" mean?
3. Name the three limits of an LLM used alone.

Chapter 6

Prompting

System and user messages

A **prompt** is what you send to the model. Chat models take a list of **messages**. The **system** message sets the role and the rules ("You build an enterprise knowledge graph... return JSON"). The **user** message carries the task and the data. The model's reply is the **assistant** message.

Asking for structured output

To make a model's answer usable by code, ask for JSON and show the exact shape:

```
Return JSON: {"intent": "qa" | "experts" | "recommend", "entities": [...]}
```

Models still sometimes wrap JSON in prose or code fences. CRAG's `parse_json` finds the first `{` or `[` and decodes only the JSON part; if that fails, `chat_json` sends the reply back once asking for "only valid JSON".

Grounding

To reduce hallucinations, the answering prompt says: answer **only** from the numbered sources, cite each claim with its number, and say so when the sources do not contain the answer. This is called **grounding**.

Self-check

1. Why does CRAG show the model the exact JSON shape it expects?
2. What is grounding?

Chapter 7

Embeddings and vector search

Meaning as a direction

An **embedding model** turns a text into a **vector**: a list of numbers (CRAG's has 2,048). It is trained so that texts with similar meaning get vectors pointing in similar directions. "Payroll processing" and "salary administration" share no word but end up close.

Measuring closeness: the cosine

Take two short vectors to see the computation: a = (1, 2, 2) and b = (2, 1, 2).

- **Dot product**: a·b = 1×2 + 2×1 + 2×2 = 8.
- **Length**: ‖a‖ = √(1+4+4) = 3, ‖b‖ = √(4+1+4) = 3.
- **Cosine similarity**: 8 / (3×3) = 0.89. Close to 1: similar.

The cosine ranges from −1 (opposite) to 1 (same direction). If every vector is first divided by its length (**normalised**), the cosine is just the dot product. CRAG normalises all vectors once, so ranking thousands of passages is one matrix multiplication.

Queries and passages

Questions are short and passages long, so modern embedding models encode them slightly differently. CRAG passes `input_type: "query"` for questions and `"passage"` for documents.

Finding nearest vectors fast: HNSW

Comparing a question with every vector is fine for thousands of passages, slow for millions. An **HNSW** index (Hierarchical Navigable Small World) builds layers of graphs over the vectors: a sparse top layer with long links and denser lower layers. Search starts at the top, jumps greedily toward the query, and goes down layer by layer, like finding a street by going from country to city to neighbourhood. Neo4j provides HNSW **vector indexes**.

In CRAG. `crag/llm.py:embed` calls `nvidia/nemotron-3.5-embed-1b` and normalises the vectors. `MemoryStore.search_chunks` ranks passages with one NumPy multiplication; `Neo4jStore.search_chunks` calls `db.index.vector.queryNodes`.

Self-check

1. Compute the cosine of (1, 0) and (0, 1). What does the result mean?
2. Why normalise vectors once at the start?

Chapter 8

Retrieval-Augmented Generation

The idea

RAG (Lewis et al., 2020) solves the first two limits of LLMs:

1. **Retrieve** the passages most relevant to the question.
2. **Augment** the prompt with these passages.
3. **Generate** an answer from them, with citations.

The model gets private, current knowledge without retraining, and every claim can be checked against a source.

Chunking

Documents are cut into **chunks** before indexing, for three reasons: an embedding represents a short text more precisely than a long one; the prompt has limited room; and a citation should point to a precise passage. CRAG packs whole sentences into chunks of at most 1,200 characters. Each new chunk repeats the last 200 characters of the previous one (**overlap**), so a fact cut at a boundary is whole in at least one chunk.

Example: a 3,000-character document gives roughly three chunks, positions 0–1,200, 1,000–2,200 and 2,000–3,000 (approximately, because chunks end at sentence boundaries).

The limits of classic RAG

Classic RAG scores each chunk **alone** against the question. Ask "Which employee knows both AutoCAD and project scheduling?": a chunk mentioning AutoCAD and another mentioning scheduling may belong to the same person, but neither is similar to the whole question, and the person is never named as such. This is the gap knowledge graphs fill.

Self-check

1. What are the three steps of RAG?
2. Why is overlap useful?
3. Give a question classic RAG answers badly, and say why.

Chapter 9

Knowledge graphs

Nodes, edges, triples

A **graph** is a set of **nodes** connected by **edges**. A **knowledge graph** uses nodes for things (**entities**: people, skills, organisations) and edges for typed **relations**. Each fact is a **triple** (subject, predicate, object):

```
(Employee 101, HAS_SKILL, AutoCAD)
(Employee 101, HAS_SKILL, Project Scheduling)
(Employee 205, HAS_SKILL, AutoCAD)
```

Now "who knows AutoCAD and scheduling?" is answered by looking at the people linked to both nodes: Employee 101.

Hops and breadth-first search

Following one edge is one **hop**. **Multi-hop** reasoning follows several: from AutoCAD (hop 0) to the people who use it (hop 1) to their other skills (hop 2). Finding everything within k hops is done with **breadth-first search** (BFS): visit all neighbours, then all neighbours of neighbours, up to depth k.

Property graphs and Neo4j

In a **property graph**, nodes have **labels** (`Document`, `Chunk`, `Entity`) and both nodes and edges carry **properties** (key–value pairs, such as `name` or `type`). **Neo4j** is the most widely used property-graph database. It is queried with **Cypher**, whose patterns look like the drawing of the graph:

```
MATCH (p:Entity {type:'Person'})-[r:RELATED]->(s:Entity {key:'autocad'})
RETURN p.name, r.type
```

`MATCH` describes a pattern; `(p:Entity)` is a node with label Entity; `-[r:RELATED]->` is an edge; `RETURN` lists the output. `MERGE` creates something only if it does not already exist, which makes repeated imports safe.

In CRAG. The graph model: `(:Document)-[:HAS_CHUNK]->(:Chunk)-[:MENTIONS]->(:Entity)-[:RELATED {type}]->(:Entity)` and `(:Entity {type:'Person'})-[:AUTHORED]->(:Document)`. See the data-model figure in the report and `crag/store.py`.

Self-check

1. Write the triple for "Alice works at Acme".
2. Starting from AutoCAD, what do you reach in 2 hops in the example above?
3. What is the difference between `MERGE` and `CREATE`?

Chapter 10

Building the graph with an LLM

Extraction

Building a graph by hand is impossible at company scale. CRAG asks the LLM, for each chunk, to list the important **entities** (with one of ten types) and the **relations** between them, as JSON. The prompt fixes the rules: allowed types, verbs in UPPER_SNAKE_CASE, both ends of each relation must be listed entities, at most 15 entities, ignore placeholders.

Never trust, always validate

Model output is checked in code:

- unknown types become `Concept`;
- relations whose source or target is not a listed entity are removed;
- known placeholders ("Company Name", "City") are removed. The test documents were anonymised with such placeholders, and the model extracted them anyway. Left in, "Company Name" would have connected every person to every other person through one hub node;
- if a call fails, that chunk contributes nothing and the run continues.

Entity resolution

The same thing is written in different ways: "Microsoft Excel", "microsoft-excel". **Entity resolution** merges them. CRAG uses a **normalised key**: lower case, punctuation replaced by spaces (keeping `+` and `#` for C++ and C#). Same key, same node. True synonyms ("MS Excel") are not merged yet; that is an improvement in `OTHER_IDEAS.md`.

Storing

Each chunk and each entity is embedded, then everything is written to the store: the document, its chunks with their vectors, the entities with their vectors, the relations with the document they came from, and the `AUTHORED` link for profile documents.

Self-check

1. Why is a placeholder entity dangerous for graph search?
2. What key does "Node.js" get after normalisation?

Chapter 11

Optical character recognition

OCR finds letters in an image. CRAG uses **Tesseract *5***, an open-source engine that locates lines of text, segments them into characters and words, and recognises them with a neural network. To OCR a PDF, each page is first **rendered** into an image (with pypdfium2, at twice the normal resolution, because OCR is more accurate on larger letters).

CRAG decides automatically: if a PDF yields fewer than 20 characters per page on average from its text layer, it is treated as scanned and OCR'd. Image files are always OCR'd.

> **In CRAG.** `crag/ingest.py:load_text`. Two scanned test documents were read correctly (about 3,900 and 5,300 characters).

Self-check

1. Why render at 2× resolution before OCR?
2. How does CRAG decide that a PDF needs OCR?

Chapter 12

GraphRAG: how CRAG retrieves

GraphRAG retrieves from a knowledge graph as well as from text. CRAG's retrieval step does five things.

1. Vector ranking

Embed the question and take the 18 most similar chunks (three times the 6 finally kept).

2. Entity linking

Find the graph nodes the question is about, the **seeds**: entities whose key equals a normalised keyword from the question, plus the 8 entities whose vectors are nearest to the question's, if their cosine is at least 0.35.

3. Graph expansion

Collect all entities within 2 hops of the seeds (at most 60, nearest first) and the relations among them. These relations become numbered **facts**, those touching a seed first.

4. Graph ranking

Rank chunks by how many of the seeds and their direct neighbours they mention.

5. Fusion with Reciprocal Rank Fusion

Two rankings, two kinds of score (cosine values, mention counts). **RRF** ignores the scores and uses only ranks:

$$RRF(chunk) = \Sigma \text{ over rankings of } 1 / (60 + rank)$$

Example. Chunk A is 1st in the vector list and absent from the graph list; chunk B is 3rd in both.

- A: 1/(60+1) = 0.0164
- B: 1/(60+3) + 1/(60+3) = 0.0317

B wins: agreement between two independent signals beats a single top rank.

CRAG keeps 6 chunks: the 3 best vector hits always, then the 3 best remaining fused chunks. The first evaluation showed why this rule is needed: with pure fusion, the single most relevant passage was sometimes pushed out by graph-linked passages, which hurt simple questions. The graph

should add evidence, never remove the best one. Each kept chunk is labelled `vector`, `graph` or `both`.

The corrective loop

Then the model **grades** the evidence: is it enough to answer? If not, it proposes a better query and retrieval runs once more, keeping the evidence already found. This idea comes from Corrective RAG (Yan et al., 2024). One retry at most keeps the cost bounded.

Self-check

1. Compute RRF for a chunk ranked 2nd in one list and 5th in the other.
2. Why use ranks rather than adding the scores?
3. What is a seed entity?

Chapter 13

LangGraph: the workflow as a state machine

State machines

A **state machine** is a set of steps (**nodes**), a shared memory (**state**), and rules for moving between steps (**edges**). A **conditional edge** chooses the next step from the state, for example "if the evidence is insufficient, go back to retrieve".

LangGraph

LangGraph is a library to declare such workflows for LLM applications. You write one Python function per node; each reads the state and returns it updated. You connect them:

```
g.set_entry_point("analyze")
g.add_edge("analyze", "retrieve")
g.add_conditional_edges("grade", after_grade,
                        ["retrieve", "experts", "recommend", "generate"])
```

CRAG's workflow

```
analyze → retrieve → grade —(weak, first time)→ retrieve
                           ├→ experts   → generate
                           ├→ recommend → generate
                           └→ generate → explain → end
```

In **vector mode** (the classic-RAG baseline) the graph steps and the grading are skipped: retrieve → generate. This switch is what makes a fair comparison possible: same models, same chunks, same prompts, only the graph differs.

Every node also appends a line to a **trace** (step, time, details), which the interface shows under "Workflow steps".

Self-check

1. What is the difference between an edge and a conditional edge?
2. Why is a fixed workflow easier to evaluate than a free agent?

Chapter 14

Experts, recommendations and explanations

Ranking experts

For "who are the experts in X?", each candidate person gets three signals, each divided by the largest value so it lies between 0 and 1:

- **graph evidence**: +1 for each relation between the person and a seed entity (Employee 12 HAS_SKILL Payroll), +0.25 for each relation to a direct neighbour of a seed;
- **coverage**: how many *different* seed entities the person is linked to (through the graph, their documents' entities, or the entity's name appearing in their best passages);
- **semantic**: the best cosine between the question and their chunks.

Score = 0.4 × graph + 0.3 × coverage + 0.3 × semantic.

Example. Person P has graph 1.0, coverage 0.5, semantic 0.8: 0.4 + 0.15 + 0.24 = **0.79**. Person Q has graph 0.2, coverage 1.0, semantic 0.9: 0.08 + 0.3 + 0.27 = **0.65**. P ranks first because the graph links them directly to the topic.

A lesson from the evaluation: the first version counted *all* the relations of a person, not only those to the question's entities. People with long documents have many relations, so they ranked high for every topic. This is called **degree bias**, and the fix is to count only evidence that concerns the question.

Recommending documents

For a document or a question, each candidate document scores 0.7 × semantic similarity + 0.3 × **Jaccard** overlap of entities. Jaccard = size of the intersection / size of the union. Documents with entities {SQL, Python, ETL} and {SQL, ETL, Tableau}: intersection 2, union 4, Jaccard 0.5.

Explaining answers

The answer prompt numbers passages `[S1]`… and facts `[F1]`…, and requires a citation for every claim. After generation, CRAG extracts the cited markers with a regular expression. The cited facts are the **reasoning path**, drawn in clay on the graph; the cited passages are the sources. Because the explanation comes from what the answer actually cited, it cannot tell a different story from the answer.

Self-check

1. Compute the expert score for graph 0.5, coverage 0.5, semantic 0.5.
2. Compute the Jaccard of {A, B, C} and {C, D}.

Chapter 15

The API and the interface

FastAPI

FastAPI turns Python functions into HTTP endpoints. Request bodies are declared as **Pydantic** models, so invalid input is rejected automatically with status 422:

```
class AskIn(BaseModel):
    question: str = Field(min_length=2, max_length=2000)
    mode: str = Field("graph", pattern="^(graph|vector)$")
```

Uvicorn is the server that runs the FastAPI application. Visiting `/docs` shows interactive documentation of every endpoint.

Security basics

- **Authentication**: uploading documents requires a secret administrator token in a header.
- **Validation**: file type allow-list, 20 MB size limit, question length limits.
- **Rate limiting**: at most 20 model-backed requests per minute per client address, to protect cost and availability.
- **Escaping**: text from documents or the model is escaped before display, so a document containing `<script>` cannot run code in the user's browser (this attack is called **XSS**).
- **Read-only mode**: the public demo refuses uploads entirely.

The web page

The interface is written in **HTML** (structure), **CSS** (appearance) and **JavaScript** (behaviour). JavaScript calls the API with `fetch`, then writes the answer, the sources and the experts into the page. **Cytoscape.js** draws the graph: nodes coloured by type, seeds ringed, cited relations in clay. The design follows Anthropic's website: ivory background, dark ink, one warm accent, a serif display font.

Self-check

1. What happens if the client sends `mode: "sql"`?
2. Why escape text before displaying it?

Chapter 16

Testing

A **unit test** checks one piece of code automatically. CRAG has 11, run with `pytest`. The difficulty: the real pipeline calls a paid, slow, non-deterministic model. The tests replace it with **fakes**: a fake chat function that returns fixed JSON, and a fake embedding that hashes words into a 64-number vector, so texts sharing words are similar. With fakes, the full workflow runs offline in about one second and always gives the same result.

The tests cover chunk sizes and overlap, normalisation, JSON parsing, graph building, saving and reloading, neighbourhoods, RRF, the complete workflow in both modes, and the API's validation and security.

Self-check

1. Why not call the real model in unit tests?
2. Name two things the tests check about the API.

Chapter 17

Evaluation

A fair comparison

To show the graph helps, CRAG is compared with a **baseline**: the same system in vector mode, i.e. classic RAG. Only the graph steps differ.

Metrics

- **Precision@5 (P@5)**: of the 5 experts returned, the fraction from the correct domain. 3 correct out of 5 gives 0.6.
- **Hit rate**: the fraction of questions whose gold document is among the retrieved passages.
- **Exact match**: whether the answer names the right employee.
- **LLM-as-a-judge**: a model scores correctness (against a reference answer) and **faithfulness** (are all claims supported by the context?) from 1 to 5. Useful but biased, so the main metrics above do not depend on it.

The three benchmarks

1. **Expert finding**: 12 questions such as "Who are the experts in payroll and employee relations?". Gold: the people whose profile belongs to that domain.
2. **Single-hop questions**: the model writes a question and answer from one random passage. This favours vector search; it checks that the graph does not hurt.
3. **Relational questions**: "Which employee is associated with both X and Y?", where each of X and Y alone matches 2 to 6 people but together exactly one. This is the case the graph is designed for.

The results and their discussion are in Chapter 5 of the report and in `data/eval/results.json`.

Self-check

1. An expert search returns 5 people, 4 from the right domain. What is P@5?
2. Why are there benchmarks that do not use an LLM judge?

Chapter 18

Git, Docker and deployment

Git and GitHub

Git records the history of the code as **commits** (snapshots with a message). **GitHub** hosts the repository online. `.gitignore` lists files never to commit: `.env` (secrets), `.venv` (installed libraries), test data.

Docker

A **container** packages an application with everything it needs (Python version, libraries, Tesseract) so it runs identically everywhere. A **Dockerfile** is the recipe for the **image**. **Docker Compose** starts several containers together; CRAG's starts two: Neo4j and the API. The API waits for Neo4j's **health check** before starting, and runs as a non-root user for safety.

```
docker compose up -d     # start Neo4j + CRAG on http://localhost:8080
```

Serverless deployment

On **Vercel**, code runs as a **serverless function** started per request, with no permanent disk and no database server. CRAG therefore serves its in-memory snapshot there, in read-only mode, while Docker remains the production setup. The API key is stored in Vercel's encrypted environment settings.

Self-check

1. Why is `.env` in `.gitignore`?
2. Why is the public demo read-only?

Chapter 19

Putting it all together

Follow one question through the whole system: *"Who are the experts in payroll and benefits administration?"*

1. The browser sends `POST /api/ask` with the question and `mode: "graph"`. FastAPI validates it and checks the rate limit.
2. **analyze**: Nemotron returns `{"intent": "experts", "entities": ["payroll", "benefits administration"]}`.
3. **retrieve**: the question is embedded (2,048 numbers). The 18 nearest chunks are found. "payroll" matches an entity key exactly; vector search adds nearby entities such as "Benefits Administration" and "HRIS". A 2-hop walk reaches the people who have these skills and their other skills; the relations become facts F1…F25. Chunks mentioning these entities form the graph ranking; RRF merges the two rankings into 6 passages.
4. **grade**: Nemotron judges the evidence sufficient.
5. **experts**: each person near those entities is scored: 0.4 graph evidence + 0.3 coverage + 0.3 semantic. The top 5 are kept with their evidence as citable facts.
6. **generate**: Nemotron writes the answer from passages S1–S6, facts F1–F25 and the ranked experts, citing each claim.
7. **explain**: the cited facts become the reasoning path.
8. The browser shows the answer with clickable citations, the experts with their score bars, the reasoning graph with the cited relations in clay, the sources and the workflow trace.

If you can explain each of these eight lines, and why each design choice was made, you understand CRAG.

Chapter 20

Glossary

Term	Meaning
API	Set of URLs a program calls to use a service
Attention	Transformer mechanism weighing every input token for each token
BFS	Breadth-first search: visit neighbours level by level
Chunk	Piece of a document (here ≤ 1,200 characters) that is embedded and retrieved
Cosine similarity	Closeness of two vectors' directions, from −1 to 1
Cypher	Neo4j's query language
Docker	Tool that packages and runs applications in containers
Embedding	Vector representing the meaning of a text
Entity	Node of the knowledge graph: a person, skill, organisation…
Entity linking	Mapping words of a question to graph nodes
Entity resolution	Merging different spellings of one thing into one node
Faithfulness	Whether an answer's claims are supported by its context
GraphRAG	RAG that also retrieves from a knowledge graph
Hallucination	Fluent but false statement by a model
HNSW	Layered graph index for fast nearest-vector search
Hop	Following one edge in a graph
Degree bias	Favouring nodes with many edges whatever the question
HTTP	Protocol of requests and responses on the web
Jaccard	Intersection size divided by union size of two sets
Knowledge graph	Entities connected by typed relations
LangGraph	Library to run LLM workflows as state machines
LLM	Large language model: predicts the next token
MoE	Mixture of experts: a model that activates only part of its parameters per token
Neo4j	Graph database used by CRAG
OCR	Reading text from images
Overlap	Text repeated between consecutive chunks
P@5	Fraction of the top 5 results that are correct

Term	Meaning
Prompt	Instructions and data sent to the model
RAG	Retrieve passages, then generate an answer from them
RRF	Reciprocal Rank Fusion: sum of 1/(60 + rank)
Temperature	Randomness of the model's token choice
Token	Piece of text a model processes
Triple	(subject, predicate, object) fact
Vector	List of numbers

Chapter 21

Answers to the self-checks

Computers, data and text. 1. A scan stores pixels, not characters; OCR must recognise them. 2. `{}` is an object of key–value pairs; `[]` is an ordered list.

Python. 1. So it is never published with the code (`.env` is git-ignored) and can differ per environment. 2. Conflicts between library versions required by different projects.

The web. 1. Invalid input; CRAG returns it for a question shorter than 2 characters or an unknown mode. 2. Retrying immediately would hit the same limit; waiting longer each time gives the service time to recover.

Machine learning. 1. Training adjusts parameters; inference uses them. CRAG only does inference. 2. How wrong the predictions are on known examples.

LLMs. 1. To get the same, parse-able JSON every time. 2. 30 billion parameters in total, about 3 billion active per token. 3. No private or recent knowledge; hallucinations; limited and imperfectly used context.

Prompting. 1. So the code can parse it reliably. 2. Instructing the model to answer only from given sources, with citations.

Embeddings. 1. 0: the vectors are perpendicular, unrelated. 2. So every later similarity is a cheap dot product.

RAG. 1. Retrieve, augment, generate. 2. A fact cut by a chunk boundary still appears whole in one chunk. 3. "Who knows both X and Y?": no single chunk is similar to the whole question.

Knowledge graphs. 1. (Alice, WORKS_AT, Acme). 2. Employee 101 and Employee 205 at hop 1; Project Scheduling at hop 2. 3. `MERGE` creates only if absent; `CREATE` always creates (duplicates on re-import).

Building the graph. 1. It connects unrelated people through one hub, so graph search returns everyone. 2. `node js`.

OCR. 1. Larger characters are recognised more accurately. 2. Fewer than 20 characters per page on average in the text layer.

GraphRAG. 1. 1/62 + 1/65 = 0.0315. 2. Cosine values and mention counts are on different scales; ranks are comparable. 3. A graph entity linked to the question, from which expansion starts.

LangGraph. 1. An edge always leads to the same node; a conditional edge chooses using the state. 2. Its steps and cost are known in advance, so runs are comparable and testable.

Experts. 1. 0.2 + 0.15 + 0.15 = 0.5. 2. Intersection {C} = 1, union {A, B, C, D} = 4: 0.25.

API. 1. FastAPI returns 422 before any model is called. 2. To prevent code hidden in a document from running in the user's browser (XSS).

Testing. 1. It is slow, costs money and gives different outputs, so tests would be flaky. 2. Invalid questions are rejected with 422; uploads without a token are refused.

Evaluation. 1. 4/5 = 0.8. 2. Judges are biased (here, same model family); objective metrics avoid that.

Deployment. 1. It holds secrets. 2. Serverless functions have no persistent disk, and public uploads would be a security risk.